

# SHT30 模拟 IIC 代码精读

结合《IIC 总线面试问答》理解：协议概念是怎么用 GPIO 一行行写出来的 · 2026-06

上一份《IIC 总线面试问答》讲的是**协议本身**——起始/停止信号、应答、数据有效性、读写流程。这一份把正点原子竞赛工程里 **SHT30 温湿度传感器**的驱动 (**sht30.c / sht30.h**) 拿出来，逐个函数讲清楚：**那些协议概念，到底是用 GPIO 翻电平怎么"模拟"出来的**。看的时候建议两份对照着读。

## 一、先看全局：这份代码在干嘛

SHT30 是一颗 I<sup>2</sup>C 接口的温湿度传感器，挂在 STM32 的两根普通 IO 上：**SDA = PB8、SCL = PB9**（见 sht30.h），器件地址 **0x44**。STM32 没有用硬件 I<sup>2</sup>C 外设，而是用 GPIO 手动翻电平来"模拟"出 I<sup>2</sup>C 时序——这正是上一份 PDF 里 Q11 说的"软件模拟 IIC"。

整份代码可以分成三层，从下往上搭起来：

- **最底层**：操作单根引脚的宏 —— SDA(x)/SCL(x) 输出高低、SDA\_GET() 读电平、SDA\_OUT()/SDA\_IN() 切换 SDA 方向。
- **中间层**：把电平动作组合成协议动作 —— IIC\_Start / IIC\_Stop / Send\_Byte / Read\_Byte / IIC\_Send\_Ack / I2C\_WaitAck。
- **最上层**：把协议动作组合成一次完整业务 —— SHT31\_Write\_mode（发命令）、SHT30\_Read（发命令 + 读 6 字节 + CRC 校验 + 换算成温湿度）。

## 二、概念 → 代码 对照表

先建立一个"地图"。上一份 PDF 的每个协议概念，在这份代码里都能找到对应的函数：

| 上一份 PDF 里的概念                  | 本代码里对应的函数 / 写法                          |
|-------------------------------|-----------------------------------------|
| 起始信号<br>(SCL 高时 SDA 由高→低)     | IIC_Start()                             |
| 停止信号<br>(SCL 高时 SDA 由低→高)     | IIC_Stop()                              |
| 数据有效性<br>(SCL 高时 SDA 必须稳定)    | Send_Byte() 里"先放 SDA，再拉高 SCL"的顺序        |
| 一个字节 8 位、MSB 先发               | Send_Byte() / Read_Byte() 的 for 循环 + 移位 |
| 第 9 个时钟的应答位 ACK/NACK          | 主机发：IIC_Send_Ack(); 主机收：I2C_WaitAck()   |
| 写操作流程<br>(S + 地址+0 + 数据 + P)  | SHT31_Write_mode()                      |
| 读操作流程<br>(S + 地址+1 + 读数据 + P) | SHT30_Read() 里读 6 字节那一段                 |
| 开漏 + 上拉、空闲为高                  | SDA_OUT/SDA_IN 方向切换（本代码用推挽，见第八节说明）      |

### 三、最底层：操作引脚的几个宏 (sht30.h)

这些宏是后面一切的基础，先看懂它们：

```
#define SDA(x) GPIO_WriteBit(PORT_SHT30, GPIO_SDA, (x ? Bit_SET : Bit_RESET)) /* SDA 输出高/低 */
#define SCL(x) GPIO_WriteBit(PORT_SHT30, GPIO_SCL, (x ? Bit_SET : Bit_RESET)) /* SCL 输出高/低 */
#define SDA_GET() GPIO_ReadInputDataBit(PORT_SHT30, GPIO_SDA) /* 读 SDA 当前电平 */
/* SDA_OUT(): 把 SDA 配成推挽输出；SDA_IN(): 把 SDA 配成输入。见附录完整宏 */
```

为什么单单 **SDA 要来回切换方向**，SCL 不用？因为 SCL 始终由主机（STM32）产生，永远是输出。而 SDA 是**双向**的：主机发命令时 STM32 要往 SDA 上写（输出），从机回 ACK 或返回数据时 STM32 要读 SDA（输入）。所以每次换方向前都得先调用 SDA\_OUT() 或 SDA\_IN()。

对应上一份 Q3：I<sup>2</sup>C 的 SDA 是双向线。硬件 I<sup>2</sup>C 靠开漏 + 上拉自动实现“谁都能拉低”；软件模拟里就靠**手动切换 GPIO 输入/输出方向**来让出或占用这根线。

### 四、起始信号与停止信号

这是上一份 Q4 的代码版。口诀还是那句：看 **SCL 高电平期间 SDA 往哪边跳**。

#### IIC\_Start() —— 起始信号

```
void IIC_Start(void)
{
    SDA_OUT();
    SCL(1);
    SDA(1); /* 先让 SDA、SCL 都为高 = 总线空闲状态 */
    delay_us(5);
    SDA(0); /* 关键：在 SCL 仍为高时，把 SDA 由高拉低 → 这就是 START */
    delay_us(5);
    SCL(0); /* 起始信号发完，拉低 SCL，准备发数据 */
}
```

注意动作顺序：**SCL 保持高**的同时，SDA 从 1 变 0。这正好就是“SCL 高电平期间 SDA 由高到低跳变”的定义。delay\_us(5) 是为了让电平稳定、满足从机识别的建立时间。

#### IIC\_Stop() —— 停止信号

```
void IIC_Stop(void)
{
    SDA_OUT();
    SCL(0);
    SDA(0); /* 先把 SDA 压低 */
    SCL(1); /* 再把 SCL 抬高 */
    delay_us(5);
    SDA(1); /* 关键：在 SCL 为高时，把 SDA 由低拉高 → 这就是 STOP */
    delay_us(5);
}
```

和 START 正好镜像：SCL 高电平期间，SDA 由低到高跳变。发完之后总线回到空闲。

### 五、发送一个字节：Send\_Byte()

上一份说"一个字节 8 位、高位 (MSB) 先发"。代码就是一个发 8 次的循环：

```
void Send_Byte(u8 dat)
{
    int i = 0;
    SDA_OUT();
    SCL(0);          /* 拉低 SCL，进入"可以改 SDA"的窗口 */
    for (i = 0; i < 8; i++)
    {
        SDA((dat & 0x80) >> 7); /* 取最高位放到 SDA (先发 MSB) */
        delay_us(1);
        SCL(1);          /* SCL 上升沿：从机此刻锁存 SDA 上的这一位 */
        delay_us(5);
        SCL(0);          /* SCL 拉低，准备改下一位 */
        delay_us(5);
        dat <<= 1;        /* 左移一位，让次高位变成新的最高位 */
    }
}
```

这里藏着上一份 Q5 讲的**数据有效性**规则，看顺序就懂了：

- **SCL 还是低**的时候才改 SDA (**SDA(...)** 那一行) ；
- 改好以后**才把 SCL 拉高**，并保持一会儿——这段高电平期间 SDA 绝不变动，从机安心采样；
- 采样完再拉低 SCL，去改下一位。

一句话："低电平改数据，高电平采数据"。违反这条（在 SCL 高时乱动 SDA）就会被从机误当成 START/STOP。这就是为什么协议要这么规定。

## 六、读取一个字节：Read\_Byte()

读是发的反过程：STM32 把 SDA 设成输入，每个 SCL 高电平期间去读一位，凑够 8 位。

```
unsigned char Read_Byte(void)
{
    unsigned char i, receive = 0;
    SDA_IN();          /* SDA 改成输入，让从机来驱动这根线 */
    for (i = 0; i < 8; i++)
    {
        SCL(0);
        delay_us(5);
        SCL(1);          /* SCL 上升沿：SDA 上的数据此时有效 */
        delay_us(5);
        receive <<= 1;    /* 先腾出最低位 */
        if (SDA_GET())
            receive |= 1; /* 读到 1 就把最低位置 1 (同样是 MSB 先收) */
        delay_us(5);
    }
    SCL(0);
    return receive;
}
```

同样是 MSB 先收：第一次读到的位最后会被移到最高位。**receive <<= 1** 配合 **|= 1** 是嵌入式里"逐位拼字节"的标准写法。

## 七、应答：主机发 ACK 与主机等 ACK

上一份 Q6 讲应答发生在**第 9 个时钟**。代码分两种角色：

## I2C\_WaitAck() —— 主机发完一字节，等从机应答

```
unsigned char I2C_WaitAck(void)
{
    char ack = 0;
    unsigned char ack_flag = 10;
    SCL(0);
    SDA(1);
    SDA_IN();          /* 主机松开 SDA（设为输入），把第 9 位让给从机 */
    SCL(1);            /* 第 9 个时钟的高电平 */
    while ((SDA_GET() == 1) && ack_flag) /* 从机若拉低 SDA = ACK；一直为高就是没应答 */
    {
        ack_flag--;
        delay_us(5);    /* 轮询等待，最多 10 次 */
    }
    if (ack_flag <= 0) { IIC_Stop(); return 1; } /* 超时没等到 ACK → 发 STOP 报错 */
    else { SCL(0); SDA_OUT(); } /* 收到 ACK，切回输出准备后续 */
    return ack;
}
```

要点：主机要“**松手**”（SDA\_IN），把 SDA 让出来，从机才能在第 9 个时钟把它拉低表示 ACK。读到低（返回 0）= 收到 ACK；一直是高 = 没人应答，超时后主动 STOP 返回 1。

## IIC\_Send\_Ack() —— 主机当接收方时，由主机回 ACK/NACK

```
void IIC_Send_Ack(unsigned char ack)
{
    SDA_OUT();
    SCL(0);
    SDA(0);
    delay_us(5);
    if (!ack) SDA(0); /* ack=0 → SDA 保持低 = ACK，告诉从机“继续发” */
    else SDA(1); /* ack=1 → SDA 拉高 = NACK，告诉从机“别发了” */
    SCL(1); /* 第 9 个时钟高电平，把应答位送出 */
    delay_us(5);
    SCL(0);
    SDA(1);
}
```

对应上一份 Q6 那个细节：**主机读最后一个字节时回 NACK**。后面 SHT30\_Read 里读第 6 个字节就是 **IIC\_Send\_Ack(1)**，前 5 个都是 **IIC\_Send\_Ack(0)**——这就是“我不要了”的信号。

## 八、串起来：一次完整的 SHT30\_Read()

现在把上面的积木拼成一次完整读取。这一段同时用到了上一份的**写操作流程**和**读操作流程**。

**第 1 步——写命令**（写操作流程：**S + 地址 + 0 + 数据 + P**）。SHT31\_Write\_mode() / SHT30\_Read() 开头都是这个套路：

```

IIC_Start();          /* S: 起始信号 */
Send_Byte((0x44 << 1) | 0); /* 地址0x44左移1位 + 最低位0(写) = 0x88 */
if (I2C_WaitAck() == 1) return 1; /* 等地址 ACK, 没应答就报错返回 */
Send_Byte(dat >> 8);      /* 命令高 8 位 */
if (I2C_WaitAck() == 1) return 2;
Send_Byte(dat & 0xff);    /* 命令低 8 位 */
if (I2C_WaitAck() == 1) return 3;

```

注意 **(0x44 << 1) | 0**: I<sup>2</sup>C 的 7 位地址要左移一位, 空出来的最低位放读写标志。这正是上一份 Q7/Q8 说的"地址 + 读写位凑成 8 位"。**0 = 写, 1 = 读**。

**第 2 步——ACK 轮询, 等传感器测完。**SHT30 收到测量命令后要花点时间出数据, 没测完它就不回 ACK。代码就靠这个特性反复试探:

```

do {
    i++;
    if (i > 20) return 4; /* 最多试 20 次 (约 40ms), 超时报错 */
    delay_ms(2);
    IIC_Start();
    Send_Byte((0x44 << 1) | 1); /* 这次最低位是 1 = 读操作 */
} while (I2C_WaitAck() == 1); /* 直到从机回 ACK (数据就绪) 才往下走 */

```

这叫 **ACK 轮询 (ACK polling)**, 是 I<sup>2</sup>C

器件常见手法: "没准备好我就不应答, 你过会儿再来问。"主机用回不回 ACK 来判断从机状态, 不用额外的忙/闲引脚。

**第 3 步——连续读 6 个字节 + 应答。**读操作流程在这里体现得最完整:

```

buff[0] = Read_Byte(); IIC_Send_Ack(0); /* 温度高字节, 回 ACK 继续 */
buff[1] = Read_Byte(); IIC_Send_Ack(0); /* 温度低字节 */
buff[2] = Read_Byte(); IIC_Send_Ack(0); /* 温度 CRC */
buff[3] = Read_Byte(); IIC_Send_Ack(0); /* 湿度高字节 */
buff[4] = Read_Byte(); IIC_Send_Ack(0); /* 湿度低字节 */
buff[5] = Read_Byte(); IIC_Send_Ack(1); /* 湿度 CRC, 回 NACK 结束! */
IIC_Stop(); /* P: 停止信号 */

```

前 5 个字节读完都回 ACK ("还要"), **第 6 个读完回 NACK ("够了")**, 然后 STOP 收尾——和上一份 Q8 的读操作流程图一字不差。

**第 4 步——CRC 校验 + 换算。**这部分已经是应用层, 不属于 I<sup>2</sup>C 协议, 但顺带说明:

- SHT30 每 2 个数据字节后跟 1 个 CRC8 校验字节, `crc8()` 重新算一遍和收到的比对, 防止线路干扰导致数据出错;
- 校验通过后按数据手册公式换算: 温度 =  $\text{raw}/65535 \times 175 - 45$  (°C), 湿度 =  $\text{raw}/65535 \times 100$  (%RH)。

## 九、几个容易被问到的细节

这份代码和"标准 I<sup>2</sup>C"有什么不一样?

它把 SDA 配成了**推挽输出** (GPIO\_Mode\_OUT + OType\_PP), 而标准 I<sup>2</sup>C 要求**开漏 + 上拉**。推挽方式靠 SDA\_OUT()/SDA\_IN() 手动切换方向来"让线", 单主单从、近距离能正常工作, 是教学/小项目里常见的简化写法。严格的多设备总线上更推荐开漏, 避免方向切换时序里两边同时驱动

。

## 为什么到处是 delay\_us(5)?

因为是软件模拟，时序完全靠延时撑出来。这些延时决定了 SCL 的高低电平宽度，也就是**等效的 I<sup>2</sup>C 速率**。把延时改小速率就高，但太小会超出 SHT30 能跟上的范围（上一份 Q10 的速率档位）。

## 为什么变量都在函数开头声明？

Keil + STM32 标准库工程默认按 C89 编译，C89

要求局部变量必须声明在语句块最前面，所以你看 **int i = 0;**

都堆在函数顶部，而不是用到时才定义。

## 附录 A: sht30.h 完整源码

```
#ifndef __SHT30_H
#define __SHT30_H

#include "stm32f4xx.h"

/* 温度和湿度全局变量，由 SHT30_Read() 更新 */
extern double Temperature, Humidity;

#define u8 unsigned char

/* 引脚定义：SHT30 挂载在 GPIOB, SDA=PB8, SCL=PB9 */
#define RCC_SHT30 RCC_AHB1Periph_GPIOB
#define PORT_SHT30 GPIOB
#define GPIO_SDA GPIO_Pin_8
#define GPIO_SCL GPIO_Pin_9

/* 将 SDA 配置为推挽输出模式（向 SHT30 发送数据/命令时使用） */
#define SDA_OUT() { \
    GPIO_InitTypeDef GPIO_InitStructure; \
    GPIO_InitStructure.GPIO_Pin = GPIO_SDA; \
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT; \
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP; \
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz; \
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL; \
    GPIO_Init(PORT_SHT30, &GPIO_InitStructure); \
}

/* 将 SDA 配置为浮空输入模式（接收数据或等待 ACK 时使用） */
#define SDA_IN() { \
    GPIO_InitTypeDef GPIO_InitStructure; \
    GPIO_InitStructure.GPIO_Pin = GPIO_SDA; \
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN; \
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP; \
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz; \
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL; \
    GPIO_Init(PORT_SHT30, &GPIO_InitStructure); \
}

/* 读取 SDA 引脚当前电平（0 或 1） */
#define SDA_GET() GPIO_ReadInputDataBit(PORT_SHT30, GPIO_SDA)

/* 控制 SDA / SCL 输出高低电平 */
#define SDA(x) GPIO_WriteBit(PORT_SHT30, GPIO_SDA, (x ? Bit_SET : Bit_RESET))
#define SCL(x) GPIO_WriteBit(PORT_SHT30, GPIO_SCL, (x ? Bit_SET : Bit_RESET))
```

```
void SHT30_GPIO_Init(void);
char SHT30_Read(uint16_t dat);
```

```
#endif
```

## 附录 B: sht30.c 完整源码

```
#include "sht30.h"
#include "stdio.h"
#include "SysTick.h"
```

```
/* 温度和湿度全局变量，由 SHT30_Read() 更新后供外部使用 */
double Temperature = 0.0, Humidity = 0.0;
```

```
/* -----
 * SHT30_GPIO_Init
 * 初始化 SHT30 使用的模拟 I2C 引脚 (PB8=SDA, PB9=SCL)
 * 配置为推挽输出，速度 100MHz，不带上下拉
 * ----- */
```

```
void SHT30_GPIO_Init(void)
```

```
{
    /* C89: 所有变量声明必须在函数块开头 */
    GPIO_InitTypeDef GPIO_InitStructure;
```

```
    RCC_AHB1PeriphClockCmd(RCC_SHT30, ENABLE); /* 使能 GPIOB 时钟 */
```

```
    GPIO_InitStructure.GPIO_Pin = GPIO_SDA | GPIO_SCL;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
    GPIO_Init(PORT_SHT30, &GPIO_InitStructure);
}
```

```
/* -----
 * IIC_Start
 * 产生 I2C 起始信号: SCL 高电平期间 SDA 由高变低
 * 起始信号后 SCL 保持低电平，等待后续数据传输
 * ----- */
```

```
void IIC_Start(void)
```

```
{
    SDA_OUT();
    SCL(1);
    SDA(1); /* 确保 SDA 为高，建立起始前的空闲状态 */
    delay_us(5);
    SDA(0); /* SDA 在 SCL 高期间下拉 → 产生 START 条件 */
    delay_us(5);
    SCL(0); /* SCL 拉低，准备后续数据传输 */
}
```

```
/* -----
 * IIC_Stop
 * 产生 I2C 停止信号: SCL 高电平期间 SDA 由低变高
 * ----- */
```

```
void IIC_Stop(void)
```

```
{
    SDA_OUT();
    SCL(0);
    SDA(0); /* 确保 SDA 为低 */
    SCL(1);
    delay_us(5);
}
```

```

    SDA(1); /* SDA 在 SCL 高期间拉高 → 产生 STOP 条件 */
    delay_us(5);
}

/* -----
 * IIC_Send_Ack
 * 主机发送应答 / 非应答信号
 * ack = 0: 发送 ACK (继续接收)  ack = 1: 发送 NACK (停止接收)
 * ----- */
void IIC_Send_Ack(unsigned char ack)
{
    SDA_OUT();
    SCL(0);
    SDA(0);
    delay_us(5);
    if (!ack) SDA(0); /* ACK: SDA 保持低 */
    else     SDA(1); /* NACK: SDA 拉高 */
    SCL(1);
    delay_us(5);
    SCL(0);
    SDA(1);
}

/* -----
 * I2C_WaitAck
 * 主机释放 SDA 后等待从机拉低 SDA 作为应答
 * 最多轮询 10 次 (每次 5us) , 超时则发送 STOP 并返回错误
 * 返回 0: 收到 ACK; 返回 1: 超时未收到 ACK
 * ----- */
unsigned char I2C_WaitAck(void)
{
    char ack = 0;
    unsigned char ack_flag = 10; /* 最大等待次数 */

    SCL(0);
    SDA(1);
    SDA_IN(); /* 切换 SDA 为输入, 读取从机应答电平 */

    SCL(1);
    while ((SDA_GET() == 1) && ack_flag)
    {
        ack_flag--;
        delay_us(5);
    }

    if (ack_flag <= 0)
    {
        IIC_Stop(); /* 超时: 主动发送停止信号 */
        return 1;
    }
    else
    {
        SCL(0);
        SDA_OUT(); /* 切回输出模式, 准备后续操作 */
    }
    return ack;
}

/* -----
 * Send_Byte
 * 向 I2C 总线发送 1 字节数据, 高位 (MSB) 先发
 * ----- */
void Send_Byte(u8 dat)

```



```

{
    int i = 0;
    SDA_OUT();
    SCL(0); /* 拉低 SCL, 开始数据传输 */

    for (i = 0; i < 8; i++)
    {
        SDA((dat & 0x80) >> 7); /* 取最高位输出到 SDA */
        delay_us(1);
        SCL(1); /* SCL 上升沿: 从机锁存当前 SDA 电平 */
        delay_us(5);
        SCL(0); /* SCL 拉低, 准备输出下一位 */
        delay_us(5);
        dat <<= 1; /* 左移, 将次高位移到最高位 */
    }
}

/* -----
* Read_Byte
* 从 I2C 总线读取 1 字节数据, 高位 (MSB) 先收
* ----- */
unsigned char Read_Byte(void)
{
    unsigned char i, receive = 0;
    SDA_IN(); /* 切换 SDA 为输入 */

    for (i = 0; i < 8; i++)
    {
        SCL(0);
        delay_us(5);
        SCL(1); /* SCL 上升沿: SDA 上的数据有效 */
        delay_us(5);
        receive <<= 1; /* 左移为新位腾出空间 */
        if (SDA_GET())
        {
            receive |= 1; /* 读取当前位 */
        }
        delay_us(5);
    }
    SCL(0);
    return receive;
}

/* -----
* SHT31_Write_mode
* 向 SHT30 (I2C 地址 0x44) 发送一条 16 位命令
* 返回 0: 发送成功; 1/2/3: 对应步骤未收到 ACK, 通信失败
* ----- */
char SHT31_Write_mode(uint16_t dat)
{
    IIC_Start();

    /* 发送器件地址: 0x44 左移1位, 末位置0表示写操作 (0x88) */
    Send_Byte((0x44 << 1) | 0);
    if (I2C_WaitAck() == 1) return 1; /* 地址阶段未收到 ACK */

    /* 发送命令高8位 */
    Send_Byte(dat >> 8);
    if (I2C_WaitAck() == 1) return 2;

    /* 发送命令低8位 */
    Send_Byte(dat & 0xff);
    if (I2C_WaitAck() == 1) return 3;
}

```

```

    return 0;
}

/* -----
 * crc8
 * CRC-8 校验 (多项式 0x31, 初值 0xFF), 符合 SHT30 数据手册规定
 * data: 待校验数据首地址; len: 数据字节数
 * 返回: 计算得到的 CRC 值, 与传感器返回的校验字节比对
 * ----- */
unsigned char crc8(const unsigned char *data, int len)
{
    const unsigned char POLYNOMIAL = 0x31;
    unsigned char crc = 0xFF;
    int j, i;

    for (j = 0; j < len; j++)
    {
        crc ^= *data++;
        for (i = 0; i < 8; i++)
        {
            /* 若最高位为1, 左移后异或多项式; 否则直接左移 */
            crc = (crc & 0x80) ? (crc << 1) ^ POLYNOMIAL : (crc << 1);
        }
    }
    return crc;
}

/* -----
 * SHT30_Read
 * 读取 SHT30 温湿度值, 结果写入全局变量 Temperature / Humidity
 *
 * dat: 读取命令
 * 周期测量模式取回数据: 0xe000 (当前使用)
 * 单次测量 (高重复性): 0x2c06
 * 单次测量 (中重复性): 0x2400
 *
 * 返回: 0 = 读取成功; 1~4 = 通信超时/错误; 5 = CRC 校验失败
 * ----- */
char SHT30_Read(uint16_t dat)
{
    uint16_t i = 0;
    unsigned char buff[6] = {0}; /* 6字节原始数据: 温度高/低/CRC, 湿度高/低/CRC */
    uint16_t data_16 = 0;

    /* 发送周期测量启动命令: 中等重复性, 1次/秒 (0x2130) */
    SHT31_Write_mode(0x2130);

    /* 发送取回数据命令 (dat = 0xe000) */
    IIC_Start();
    Send_Byte((0x44 << 1) | 0);
    if (I2C_WaitAck() == 1) return 1;
    Send_Byte(dat >> 8);
    if (I2C_WaitAck() == 1) return 2;
    Send_Byte(dat & 0xff);
    if (I2C_WaitAck() == 1) return 3;

    /* 轮询等待 SHT30 完成测量并就绪
     * 每隔 2ms 重新发送读地址; ACK=0 表示数据就绪
     * 最多等待 20 次 (约 40ms), 超时返回错误 4
     */
    do
    {

```

```

i++;
if (i > 20) return 4;
delay_ms(2);

IIC_Start();
Send_Byte((0x44 << 1) | 1); /* 发送读地址（末位1=读操作） */

} while (I2C_WaitAck() == 1);

/* 依次读取 6 字节原始数据 */
buff[0] = Read_Byte(); IIC_Send_Ack(0); /* 温度高8位 */
buff[1] = Read_Byte(); IIC_Send_Ack(0); /* 温度低8位 */
buff[2] = Read_Byte(); IIC_Send_Ack(0); /* 温度 CRC 校验值 */
buff[3] = Read_Byte(); IIC_Send_Ack(0); /* 湿度高8位 */
buff[4] = Read_Byte(); IIC_Send_Ack(0); /* 湿度低8位 */
buff[5] = Read_Byte(); IIC_Send_Ack(1); /* 湿度 CRC 校验值（NACK 结束读取） */

IIC_Stop();

/* CRC 校验：对温度2字节计算CRC与buff[2]比对，对湿度2字节与buff[5]比对 */
if ((crc8(buff, 2) == buff[2]) && (crc8(buff + 3, 2) == buff[5]))
{
    /* 温度转换公式：T(°C) = (raw / 65535) × 175 - 45 */
    data_16 = (buff[0] << 8) | buff[1];
    Temperature = (data_16 / 65535.0) * 175.0 - 45;

    /* 湿度转换公式：RH(%RH) = (raw / 65535) × 100 */
    data_16 = (buff[3] << 8) | buff[4];
    Humidity = (data_16 / 65535.0) * 100.0;

    return 0;
}
else
{
    printf("CRC check failed\r\n");
}
return 5;
}

```